

Node and Express Login App

Overview

This is a small Node.js and Express application for employee management with a simple authentication flow. It lets users sign up, sign in, view an employee dashboard, inspect individual employee records, and perform basic create, update, and delete actions.

The app is designed as a lightweight demo project. Instead of using a database, it stores users and employee records in local JSON files, which keeps the setup simple and makes the project easy to run in a classroom, portfolio, or prototype environment.

Utility

The app is useful as a compact example of how to build a full web workflow with Express and server-side rendering. It demonstrates:

- User registration and login
- Password hashing and verification
- Employee listing with filtering and sorting
- Employee detail views
- Add, edit, and delete forms
- Simple file-based persistence

Because the data is stored in JSON files, the project is easy to understand and quick to start, while still showing the main patterns used in real CRUD applications.

User Experience

The interface is focused on clarity and speed.

- The login and sign-up pages use a polished card-based layout with strong contrast and clear form fields.
- The dashboard presents employees in a table with search, filter, and sort controls.
- Each employee row includes direct actions for viewing, editing, and deleting.
- The employee detail page summarizes one record at a time and provides action buttons for the next step.
- The add and update pages use structured forms with consistent spacing and a shared visual style.
- The delete page adds a warning and confirmation step so the user can review the record before removal.

Overall, the UX is simple and task-oriented, with most interactions kept to one or two clicks.

Technologies Used

The application uses the following technologies:

- Node.js: runtime environment for the server
- Express: routing, request handling, and middleware
- EJS: server-side templating for rendering pages
- bcryptjs: password hashing and password verification
- fs: built-in Node module for reading and writing JSON files
- path: built-in Node module for safe file path handling
- nodemon: development tool for auto-restarting the server during changes
- HTML and CSS: used inside EJS templates for page layout and styling

Internal Structure

The project has a very small and direct structure:

- app.js: main server file, route definitions, data helpers, and business logic
- employees.json: local employee data store
- logins.json: local user credential store
- views/: EJS templates for the UI

View Templates

- login.ejs: sign-in screen
- signUp.ejs: registration screen
- dashboard.ejs: employee dashboard with filters, sorting, and actions
- viewEmployee.ejs: single employee detail page
- addEmployee.ejs: employee creation form
- updateEmployee.ejs: employee update form
- deleteEmployee.ejs: delete confirmation screen

Data Flow

The app follows a straightforward server-rendered flow:

1. A user opens the login page or sign-up page.
2. Credentials are checked against logins.json using bcrypt hashing.
3. After login, the dashboard loads employee data from employees.json.
4. The dashboard applies search, filter, and sort logic on the server.
5. Create, update, and delete actions modify the employee array and write it back to the JSON file.
6. The user is redirected back to the dashboard with a success message.

Notes

- The app uses a default admin code for sign-up, configurable through the ADMIN_CODE environment variable.
- Employee names are stored in Lastname, Firstname format.
- This project is best suited for demos, training, and lightweight internal tools rather than production use.

This export is intended as a printable README snapshot of the current project description.